

DIGITAL TWIN CATALOG

API and Frontend Connector

Read-only REST API, tile to database link, Cesium viewer and Postman collection.

Project	Digital Twin Catalog (NARAGA 3D Catalog), reusable 3DCityDB template
Document	API and Connector
Version	0.1 draft for review, 4 October 2026
Author	Fairuz Akmal Pradana
Repository	github.com/fakmalpradana/naraga-twin-db

Contents

Introduction

Endpoints

Tile ↔ database link

Connector deliverables

Introduction

Status: **draft v0.1 for review** · Base path `/api/v1` · Read-only · JSON · OpenAPI at `/api/v1/openapi.json`,
UI at `/api/v1/docs` (django-ninja).

Endpoints

Endpoint	Purpose
GET /health	Service + database status
GET /catalog? lod=&theme=&dataset=&bbox=minx,miny,maxx,maxy&status=published&limit=&offset=	Layers with active tileset (source: catalog.v_layer)
GET /layers/{id}/tileset	Active tileset of a layer
GET /datasets , GET /datasets/{code}	Datasets and their layers per LOD
GET /datasets/{code}/stats	Object counts per class/LOD
GET /features/{objectid}	Attributes of one city object from citydb (name, class, height, function, usage, generic attributes, LODs available, dataset, lineage)
GET /features?dataset=&bbox=&q=&limit=	GeoJSON footprints + key attributes for search

Rules: `bbox` is lon/lat EPSG:4326; `lod` 0–4, empty = all; `status` default `published` ; pagination `limit` (default 50, max 500) + `offset` ; unknown filter value → 422.

Error format (all errors):

```
{ "error": { "code": "invalid_parameter", "message": "lod must be between 0 and 4", "details": {"field": "lod"} } }
```

Catalog item:

```
{ "layerId": "uuid", "dataset": {"code": "oikn", "name": "OIKN", "generatedBy": "external", "attribution": null}, "theme": "building", "lod": 1, "status": "published", "featureCount": 0, "isStale": false, "bbox": [116.6, -1.0, 116.9, -0.8], "tileset": {"version": 1, "provider": "cesium_ion", "ionAssetId": 123456, "url": null, "heightOffsetM": 0} }
```

(values illustrative)

Tile ↔ database link

Each tile object must carry its `objectId` (gml:id). The frontend reads it on click and calls `/features/{objectId}`. Verified status: **pending** (convertwin id preservation is checked on 9 Oct).

Connector deliverables

- `client/naraga-catalog.js` - ES module, no dependencies: `listLayers({lod})`, `loadTileset(viewer, lod)`, `getFeature(id)`, `enablePicking(viewer)`.
- `viewer/index.html` - simple Cesium viewer served at `/viewer/`: API URL + ion token inputs (token only in browser `localStorage`), LOD/theme/dataset selectors, layer metadata (status, version, stale), click → attributes, error banner.
- `postman/` - `naraga-catalog.postman_collection.json`, `local` and `railway` environments (`baseUrl`, `secret ionToken`). Tests per request: status code, schema, filter respected, pagination, error format, response-time budget. Run headless: `newman run postman/naraga-catalog.postman_collection.json -e postman/local.postman_environment.json`.

Cesium ion token lives only in the frontend. NARAGA token auth is a later phase.